Git ⭐ ⭐ ⭐ — Important Interview Questions & Answers

1.What is Git?
Git is a distributed version control system used to track changes in source code and manage different versions of a project.
Developers can work on the same project, create branches, commit changes, and collaborate without losing previous versions.
Simple ga gurthupettuko:
Git = Code changes ni track + manage cheyyadaniki version control tool.
Developer mistake chesthe previous version ki velladaniki kuda Git help chestundi.
Eng: Git is a distributed version control system used to track and manage source code changes.

2.What is Version Control?
Version control is a system used to track changes made to files over time.
It allows developers to see what changed, who changed it, and when it was changed. It also helps restore previous versions when required.
Simple ga gurthupettuko:
Version Control = Code history maintain cheyyadam.
Old code kavali ante previous version ki return avvachu.
Eng: Version control tracks changes to files and maintains their history.

3.What is the difference between Git and GitHub?
Git is a version control tool that runs on a local system.
GitHub is a cloud-based platform used to host Git repositories and collaborate with other developers.
Simple ga gurthupettuko:
Git = Tool
GitHub = Git repositories ni host/collaborate cheyyadaniki platform.
Eng: Git is a version control system, while GitHub is a platform for hosting and collaborating on Git repositories.

4.What is a Git Repository?
A Git repository is a storage location where Git tracks the files, commits, branches, and history of a project.
A repository can exist locally on a developer's machine or remotely on platforms such as GitHub.
Simple ga gurthupettuko:
Repository = Project + Git history store ayye place.
Eng: A Git repository stores project files along with their Git history.

5.What is the difference between Local and Remote Repository?
A local repository exists on your local machine.
A remote repository is hosted on a remote server or platform such as GitHub and is used for collaboration and sharing code.
Simple ga gurthupettuko:
Local = Mana system lo
Remote = GitHub/server lo
Eng: A local repository is on your machine, while a remote repository is hosted on a remote server.

6.What is git init**?**
git init is used to create a new Git repository in a directory.
After running this command, Git starts tracking the directory and creates the hidden .git directory containing Git metadata.
Simple ga gurthupettuko:
git init = Existing folder ni Git repository ga convert cheyyadam.
Eng: git init initializes a new Git repository in a directory.

7.What is git clone**?**
git clone is used to create a local copy of an existing remote Git repository.
It downloads the project files along with the repository history and configuration.
Simple ga gurthupettuko:
git clone = Remote repository ni local system ki copy cheyyadam.
Eng: git clone creates a local copy of a remote repository.

8.What is git status**?**
git status shows the current state of the working directory and staging area.
It tells us which files are modified, untracked, staged, or ready to be committed.
Simple ga gurthupettuko:
git status = Ippudu mana changes situation enti ani check cheyyadam.
Eng: git status shows the current state of tracked, modified, and staged files.

9.What is git add**?**
git add is used to move changes from the working directory to the staging area.
It tells Git which changes we want to include in the next commit.
Simple ga gurthupettuko:
git add = Commit cheyyaboye changes ni staging area lo pettadam.
Eng: git add moves selected changes to the staging area for the next commit.

10.What is the Git Staging Area?
The staging area is an intermediate area between the working directory and the Git repository.
It allows us to select exactly which changes should be included in the next commit.
Simple ga gurthupettuko:
Working Directory → Staging Area → Commit
Eng: The staging area is used to prepare selected changes before committing them.

11.What is git commit**?**
git commit saves the staged changes into the local Git repository as a new version.
Each commit contains information about the changes and normally has a commit message explaining what was changed.
Simple ga gurthupettuko:
git commit = Staged changes ni Git history lo save cheyyadam.
Eng: git commit records staged changes in the local Git repository.

12.What is a Commit?
A commit is a snapshot of changes saved in the Git repository.
It represents a specific point in the project's history and can be used to understand or restore the code state at that point.
Simple ga gurthupettuko:
Commit = Code history lo oka saved point.
Eng: A commit is a saved snapshot of changes in Git history.

13.What is a Git Branch?
A branch is an independent line of development in Git.
Developers commonly create branches for features or bug fixes so that changes can be developed separately without directly affecting the main branch.
Simple ga gurthupettuko:
Branch = Separate ga development cheyyadaniki path.
Eng: A branch is an independent line of development used to work on changes separately.

14.Why do we use Git branches?
Branches allow developers to work on different features or fixes independently.
For example, one developer can work on a new feature while another works on a bug fix without disturbing the main branch.
Simple ga gurthupettuko:
Feature separate → Bug fix separate → Main code safe.
Eng: Branches allow developers to work independently without directly affecting the main code.

15.What is the main/master branch?
The main branch is generally the primary branch of a repository.
It usually contains stable or production-ready code, depending on the team's development process.
Simple ga gurthupettuko:
main = Primary branch.
Production ki deploy chese stable code usually ikkada maintain chestaru.
Eng: The main branch is the primary branch used to maintain the main codebase.

16.What is git checkout**?**
git checkout can be used to switch between branches or restore certain files in Git workflows.
In modern Git, git switch is preferred specifically for switching branches, while git restore is preferred for restoring files.
Simple ga gurthupettuko:
checkout = Branch switch / old Git workflows lo restore.
Eng: git checkout can switch branches and is also used in some file restoration workflows.

17.What is git switch**?**
git switch is a Git command specifically designed to switch between branches.
It makes the intention clearer than using git checkout for branch switching.
Simple ga gurthupettuko:
git switch = Branch change cheyyadaniki.
Eng: git switch is used to switch between Git branches.

18.What is git pull**?**
git pull downloads changes from a remote repository and integrates them into the current local branch.
It is commonly used to get the latest changes from the remote branch before starting or continuing development.
Simple ga gurthupettuko:
git pull = Remote changes ni local ki techukoni integrate cheyyadam.
Eng: git pull fetches remote changes and integrates them into the current branch.

19.What is git push**?**
git push uploads local commits to a remote repository.
It is used to share locally committed changes with other developers or with the remote repository.
Simple ga gurthupettuko:
git push = Local commits → Remote repository.
Eng: git push uploads local commits to a remote repository.

20.What is git fetch**?**
git fetch downloads the latest changes and references from a remote repository without automatically merging them into the current branch.
This allows us to review remote changes before deciding whether to merge them.
Simple ga gurthupettuko:
fetch = Changes techuko, merge automatic ga cheyyadu.
Eng: git fetch downloads remote changes without merging them into the current branch.

21.What is the difference between git fetch and git pull?
git fetch downloads remote changes but does not merge them automatically.
git pull downloads remote changes and then integrates them into the current branch.
Simple ga gurthupettuko:
fetch = Download only
pull = Download + Integrate
Eng: Fetch downloads remote changes, while pull downloads and integrates them.

22.What is Git Merge?
Git merge combines changes from one branch into another branch.
For example, after completing a feature, we can merge the feature branch into the main branch.
Simple ga gurthupettuko:
Merge = Two branches changes ni combine cheyyadam.
Eng: Git merge combines changes from one branch into another branch.

23.What is a Merge Conflict?
A merge conflict occurs when Git cannot automatically combine changes from different branches.
This commonly happens when different developers modify the same part of a file.
Simple ga gurthupettuko:
Same code area ni different branches change chesthe → Conflict ravachu.
Eng: A merge conflict occurs when Git cannot automatically combine conflicting changes.

24.How do you resolve a Git merge conflict?
First, I identify the files having conflicts using git status.
Then I manually review the conflicting sections, keep the correct changes, stage the resolved files, and complete the merge commit.
Simple ga gurthupettuko:
Conflict → Check → Edit → Add → Commit
Eng: I resolve conflicts by reviewing the conflicting code, fixing it, staging the files, and completing the merge.

25.What is Git Rebase?
Rebase moves or reapplies commits from one branch on top of another branch.
It can create a cleaner, more linear project history compared with some merge-based workflows.
Simple ga gurthupettuko:
Rebase = Mana branch commits ni latest base meeda replay cheyyadam.
Eng: Rebase reapplies commits on top of another base branch to create a cleaner history.

26.What is the difference between Merge and Rebase?
Merge combines branches and creates a merge commit when necessary.
Rebase moves commits onto a new base and can create a linear history, but it rewrites commit history.
Simple ga gurthupettuko:
Merge = History preserve + combine
Rebase = History rewrite + linear
Eng: Merge combines histories, while rebase rewrites commits onto a new base.

27.What is git stash**?**
git stash temporarily saves uncommitted changes so that we can work on another branch or task without committing incomplete work.
Later, the saved changes can be reapplied.
Simple ga gurthupettuko:
stash = Temporary ga changes side lo save cheyyadam.
Eng: git stash temporarily stores uncommitted changes so they can be reapplied later.
28.When would you use Git stash?
Suppose I am working on a feature but suddenly need to switch to another branch for an urgent bug fix.
Instead of committing incomplete changes, I can stash them, switch branches, complete the urgent work, and later restore my changes.
Simple ga gurthupettuko:
Incomplete work → stash → urgent work → back → restore
Eng: I use stash when I need to temporarily save unfinished changes before switching tasks.

29.What is git reset**?**
git reset is used to move the current branch reference to another commit and can also modify the staging area or working tree depending on the reset mode.
Common modes are --soft, --mixed, and --hard.
Simple ga gurthupettuko:
reset = Previous commit state ki move cheyyadaniki use chestham.
But --hard carefully use cheyyali because uncommitted changes can be discarded.
Eng: git reset moves the branch to another commit and its effect depends on the reset mode.

30.What is git revert**?**
git revert creates a new commit that reverses the changes introduced by an earlier commit.
It is generally safer for shared branches because it does not remove existing commit history.
Simple ga gurthupettuko:
revert = Old commit ni delete cheyyadu; opposite changes tho new commit create chestundi.
Eng: git revert creates a new commit that reverses the changes of an earlier commit.

31.What is the difference between git reset and git revert?
git reset moves the branch pointer and can rewrite local history depending on how it is used.
git revert creates a new commit that undoes an earlier commit, so it is generally preferred for shared branches.
Simple ga gurthupettuko:
reset = History ni move/change cheyyachu
revert = New commit create chesi change undo chestundi
Eng: Reset can rewrite history, while revert safely creates a new commit to undo changes.

32.What is git cherry-pick**?**
git cherry-pick applies the changes introduced by a specific commit onto the current branch.
It is useful when we need one particular change from another branch without merging the entire branch.
Simple ga gurthupettuko:
Cherry-pick = Full branch vaddu, oka specific commit matrame kavali.
Eng: Cherry-pick applies a specific commit from another branch to the current branch.

33.What are Git Tags?
Git tags are references used to mark specific commits, commonly for releases or important versions.
For example, a team can create tags such as v1.0 or v2.0 to identify release points.
Simple ga gurthupettuko:
Tag = Important commit ki name/label.
Mostly **release versions** identify cheyyadaniki use chestham.
Eng: Git tags mark specific commits, commonly to identify software releases.

34.What is .gitignore**?**
.gitignore is a file used to specify files and directories that Git should not track.
It is commonly used for temporary files, build output, logs, IDE files, and sensitive local configuration files.
Simple ga gurthupettuko:
.gitignore = Git lo track cheyyakudadhu anukunna files list.
Eng: .gitignore specifies files and directories that Git should not track.

35.What is HEAD in Git?
HEAD is a reference that points to the commit or branch currently checked out in the working repository.
Normally, HEAD points to the current branch, which in turn points to the latest commit on that branch.
Simple ga gurthupettuko:
HEAD = Currently manam ekkada/which commit or branch lo unnamo indicate chestundi.
Eng: HEAD points to the currently checked-out branch or commit in Git.

36.What is a detached HEAD state?
A detached HEAD state occurs when HEAD points directly to a commit instead of a branch.
We can inspect or test that commit, but new commits made there are not automatically associated with a branch unless we create or switch to one.
Simple ga gurthupettuko:
HEAD → Direct commit
Branch ki attached ga lekapothe **detached HEAD**.
Eng: Detached HEAD means HEAD points directly to a commit instead of a branch.

37.What is Git log?
git log is used to view the commit history of a repository.
It shows information such as commit ID, author, date, and commit message.
Simple ga gurthupettuko:
git log = Commit history chudali ante.
Eng: git log displays the commit history of a Git repository.

38.How do you find which changes were made in a commit?
We can use commands such as git show to inspect the changes introduced by a specific commit.
It helps us understand what files and lines were modified in that commit.
Simple ga gurthupettuko:
git show = Specific commit lo em change ayyindo chudali ante.
Eng: I use git show to inspect the changes introduced by a specific commit.

39.What is a remote in Git?
A remote is a reference to a remote repository where Git can send or receive changes.
Common remote names include origin, which is usually created automatically when cloning a repository.
Simple ga gurthupettuko:
Remote = Remote repository address/reference.
origin = Common default remote name.
Eng: A Git remote is a reference to a remote repository used for pushing and fetching changes.

40.What is the difference between origin and upstream?
origin is commonly the default name for the remote repository from which a repository was cloned.
upstream is a commonly used name for the original or central repository when working with forks.
Simple ga gurthupettuko:
origin = Mana configured remote
upstream = Original/main project remote
Mostly fork-based workflows lo upstream concept ekkuva use chestaru.
Eng: Origin usually refers to our main remote, while upstream commonly refers to the original repository in fork-based workflows.

41.What is a Pull Request?
A Pull Request is a request to merge changes from one branch into another branch, usually through a code-hosting platform such as GitHub.
It allows developers to review code, discuss changes, run checks, and approve the changes before merging.
Simple ga gurthupettuko:
PR = "Naa branch changes ni main branch lo merge cheyyandi" ani request.
Eng: A Pull Request is a request to review and merge changes from one branch into another.

42.What is the typical Git workflow?
A common workflow is:
Clone → Create Branch → Modify Code → git add → git commit → git push → Pull Request → Review → Merge
This workflow helps teams develop features separately and safely integrate changes into the main branch.
Simple ga gurthupettuko:
Clone → Branch → Code → Add → Commit → Push → PR → Merge
Eng: A common Git workflow is to create a branch, commit changes, push them, create a PR, review, and merge.

43.What is a Git workflow used in DevOps?
In DevOps, developers usually push code to a shared Git repository.
A CI/CD pipeline can then automatically detect the change, build the application, run tests, perform code quality checks, create artifacts, and deploy the application.
Simple ga gurthupettuko:
Developer → Git → CI/CD → Build → Test → Deploy
Eng: Git acts as the source control system that triggers CI/CD workflows after code changes.

44.Why is Git important in DevOps?
Git provides version control and acts as the source of truth for application code and configuration.
It integrates with CI/CD tools such as Jenkins and GitHub Actions, allowing automated builds, testing, and deployments whenever changes are pushed.
Simple ga gurthupettuko:
Git = DevOps pipeline ki source code starting point.
Eng: Git is important in DevOps because it provides version control and integrates with CI/CD automation.

45.What is Git conflict and how can you prevent it?
A conflict occurs when Git cannot automatically combine changes, commonly because multiple developers modify the same lines.
We can reduce conflicts by frequently pulling the latest changes, keeping branches focused and short-lived, communicating with the team, and regularly integrating changes.
Simple ga gurthupettuko:
Frequent Pull + Small Branches + Regular Merge/Rebase = Fewer Conflicts
Eng: I reduce Git conflicts by keeping branches small, syncing frequently, and integrating changes regularly.